

SmartBank: The Definitive Viva Guide

An exhaustive, conversational guide to Web Architecture, Database Integrity, and Full-Stack Implementation.

Chapter 1: The Foundations of the Web (HTML & CSS)

1.1 Introduction to the Web: Clients and Servers

Before we look at a single line of code, you must understand how the internet actually works. When you open a browser like Google Chrome or Microsoft Edge, your computer acts as a **Client**. When you type in a URL (like `www.smartbank.com`), your browser doesn't magically know what to display. Instead, it sends an **HTTP Request** over the internet.

This request travels through miles of fiber optic cables until it reaches another computer, usually sitting in a massive, air-conditioned warehouse. This computer is called a **Server**. Why? Because its only job in life is to *serve* data to clients. In the SmartBank project, your Python code (using Flask) is the Server, and the browser running your HTML/JS is the Client.

The Server receives the request, processes it, and sends back an **HTTP Response**. This response contains three main ingredients: HTML (the structure), CSS (the paint), and JavaScript (the muscle). Let's dive into the first two.

1.2 HTML: The Skeleton of the Web

HTML stands for **HyperText Markup Language**. Notice that it is *not* a programming language. You cannot write logic in HTML. You cannot say, "If the user is logged in, show this button." HTML is purely a markup language used to define the structure of a document.

Tags and Elements

HTML is written using **tags**. Think of tags like containers. You open a container, put something inside, and then close it.

```
<h1>Welcome to SmartBank</h1>

<p>Your money is safe with us.</p>
```

In the example above, `<h1>` is the opening tag, and `</h1>` is the closing tag. Everything together (the tags and the text inside) is called an **HTML Element**.

The DOM (Document Object Model)

This is a crucial concept for your viva. When the browser downloads your HTML file, it doesn't just read it like a book. It converts it into a giant family tree called the **Document Object Model (DOM)**.

Imagine a real tree. The root is the `<html>` tag. The branches are the `<head>` and `<body>` tags. The leaves are the text inside your paragraphs. We care about the DOM because later, we use JavaScript to climb this tree and change the leaves without reloading the page!

Semantic HTML

In the old days, developers used `<div>` tags for absolutely everything. A header was a `div`, a footer was a `div`, a sidebar was a `div`. This is terrible for two reasons:

1. **Accessibility:** Blind users use "Screen Readers" to browse the web. If everything is a `<div>`, the screen reader doesn't know where the main content starts and where the navigation menu ends.
2. **SEO (Search Engine Optimization):** Google's robots scan websites to understand them. They look for meaning.

In SmartBank, we use **Semantic HTML**. Instead of `<div id="nav">`, we use the actual `<nav>` tag. We use `<header>`, `<main>`, `<section>`, and `<footer>`. This gives our HTML structural meaning.

1.3 CSS: The Paint and the Layout

CSS stands for **Cascading Style Sheets**. If HTML is the skeleton, CSS is the skin, clothes, and makeup. Without CSS, every website would look like a boring Word document from 1995.

Selectors and Specificity

To style an HTML element, you have to "select" it. There are three main ways to do this:

- **Tag Selector:** `p { color: red; }`. This selects *every single paragraph* on the page. It's very broad.
- **Class Selector:** `.btn { color: blue; }`. Notice the dot (.). A class is a reusable label you can stick on multiple elements. You might have 20 buttons on your page, and they all use the `class="btn"`.
- **ID Selector:** `#header { background: black; }`. Notice the hashtag (#). An ID is a unique serial number. You can only use it **once** per page.

If you give an element both a class that says "be blue" and an ID that says "be red", the ID wins. This is called **Specificity**. IDs are more specific than classes, so they override them.

The Box Model

If you learn nothing else about CSS, learn this: **Every single element on a webpage is a rectangular box**. Even if a button looks round because you gave it curved corners, the browser calculates it as a box. This is the Box Model.

The Box Model consists of four layers, starting from the inside out:

1. **Content:** The actual text or image (e.g., the word "Submit").
2. **Padding:** The invisible cushioning *inside* the box. If you add padding, the distance between the text and the border increases.
3. **Border:** The visible line that wraps around the padding.
4. **Margin:** The invisible forcefield *outside* the box. If you add margin, it pushes other boxes away from this one.

Flexbox vs. CSS Grid

In SmartBank, you used modern CSS layout tools to position things on the screen.

Flexbox (display: flex): Imagine you have 5 buttons and you want them to sit in a perfectly straight line, evenly spaced out. Flexbox is designed for 1-Dimensional layouts (either a row OR a column). It makes elements "flexible" so they stretch or shrink to fill the available space.

CSS Grid (display: grid): Imagine a chessboard. You have both rows AND columns. Grid is designed for 2-Dimensional layouts. In your SmartBank dashboard, you have a grid of cards (Total Balance, Active Loans, etc.). Grid allows you to define exactly how many columns you want (e.g., `grid-template-columns: 1fr 1fr 1fr` creates 3 equal columns).

Responsive Design and Media Queries

How does SmartBank look good on both a massive 32-inch monitor and a tiny 5-inch iPhone? Through **Media Queries**.

```
@media (max-width: 768px) {  
  
    .dashboard-grid {  
  
        grid-template-columns: 1fr;  
  
    }  
  
}
```

This code tells the browser: "If the screen width ever drops below 768 pixels, change the dashboard grid to have only 1 column." This forces the cards to stack vertically on top of each other, making them easy to scroll through on a phone.

Chapter 2: JavaScript (The Muscle and Behavior)

2.1 What is JavaScript?

If HTML is the skeleton and CSS is the paint, JavaScript (JS) is the muscle. It is the programming language of the browser. Without JS, a webpage is just a static poster. You can look at it, but it doesn't do anything. With JS, you can click buttons, animate charts, and send money across the internet without ever refreshing the page.

JavaScript runs **client-side**. This means the code is executed directly by the user's computer (inside Chrome, Safari, etc.), utilizing their CPU and RAM, rather than the server's resources.

2.2 Variables: The Memory Boxes

A variable is simply a named box in the computer's memory where you can store data. Over the years, JavaScript has changed how we create these boxes. You must know the difference for your viva.

- **var (The Old Way):** Before 2015, this was the only way to declare variables. The problem with var is that it "leaks". If you create a var inside an if statement, it can accidentally be accessed outside that if statement, causing massive, confusing bugs. Professional developers avoid var today.

- **let (The Flexible Way):** Introduced to fix var. A let variable is *block-scoped*. This means it only exists inside the curly braces {} where it was created. You can change (reassign) the value of a let variable at any time.
- **const (The Strict Way):** Short for constant. Like let, it is block-scoped. However, once you assign a value to a const, you can **never** reassign it. You should use const by default for everything, only switching to let if you know the value needs to change later (like a counter).

2.3 Data Types and Structures

JavaScript needs to know what kind of data it's holding in those variables.

- **Strings:** Text. Wrapped in quotes. "SmartBank"
- **Numbers:** Integers and decimals. 42 or 3.14
- **Booleans:** True or False. Used for logic checks. isLoggedIn = true
- **Arrays:** A list of items, ordered by an index starting at 0. const users = ["Salman", "Admin"]. To get "Salman", you ask for users[0].
- **Objects:** The most important structure in JS. An object is a collection of Key-Value pairs, wrapped in curly braces. Sound familiar? It looks exactly like a Python Dictionary!

```
const user = { name: "Salman", role: "Admin" };
```

2.4 Functions and Scope

A function is a reusable block of code designed to perform a single task. You write it once, and you can "call" it as many times as you want.

```
function calculateInterest(principal, rate) {  
  
    return principal * rate;  
  
}
```

Scope refers to where a variable is visible. If you create a variable *inside* a function, it is trapped in that function. The rest of your code has no idea it exists. This is called Local Scope, and it prevents variables from colliding with each other.

2.5 The DOM and Event Listeners

We mentioned the DOM (Document Object Model) in Chapter 1. It is the tree structure of your HTML. JavaScript's superpower is its ability to manipulate this tree in real-time.

Selecting Elements

To change an HTML element, JS must first find it. We use methods like:

- `document.getElementById('balanceDisplay')`: Finds the one specific element with that ID.
- `document.querySelector('.card')`: Finds the first element that has the class "card".

Changing Elements

Once you grab an element, you can modify it using `.innerHTML` or `.textContent`. For example, if you fetch a user's balance from the server, you can instantly display it by writing:

```
document.getElementById('balance').textContent = '₹5,000';
```

Event Listeners

A webpage is useless if it doesn't respond to the user. We use Event Listeners to "listen" for user actions (clicks, typing, scrolling).

```
const button = document.getElementById('submitBtn');

button.addEventListener('click', function() {

    alert('Money transferred successfully!');

});
```

2.6 Asynchronous JavaScript (Crucial for Viva)

This is arguably the most important JavaScript concept you need to know for a full-stack project like SmartBank.

JavaScript is **Single-Threaded**. Think of a restaurant with only one waiter. That waiter has to take orders, bring food, and collect bills. If the waiter goes to the kitchen to wait for a steak to cook, no other tables get served. The entire restaurant freezes.

In code, if JavaScript asks the Python server for a list of 10,000 transactions, it takes time over the internet. If JS simply waited, the browser would completely freeze. The user couldn't scroll, couldn't click, couldn't do anything.

Promises and Async/Await

To solve this, JS uses **Asynchronous** code. We use a feature called a Promise. A Promise is exactly what it sounds like: JS says, "I promise to go get this data, but I'll let you know when I'm done."

In modern code, we write this using the `async` and `await` keywords.

```
async function getUserData() {  
  
    console.log("Fetching data...");  
  
    const response = await fetch('/api/user/profile');  
  
    const data = await response.json();  
  
    console.log("Data received!", data);  
  
}
```

When JS hits the `await` keyword, it effectively tells the browser: "Send this network request, but don't freeze. Go ahead and let the user keep clicking and scrolling. Whenever the server finally replies, come exactly back to this line of code and continue." This creates a smooth, lag-free user experience.

JSON (JavaScript Object Notation)

You'll notice `response.json()` in the code above. JSON is the universal language of the internet. Python and JavaScript are completely different languages. They cannot read each other's code. But they both understand text.

JSON takes a JavaScript Object or a Python Dictionary and converts it into pure, plain text. The Python server sends this text over the internet. When it arrives in the browser, JS parses the text back into a usable JavaScript Object.

Chapter 3: Python (The Brain of the Backend)

3.1 Why Python?

Python is known for its incredibly clean and readable syntax. Unlike languages like Java or C++ that require massive amounts of "boilerplate" code just to print a simple word, Python gets straight to the point. In SmartBank, Python is the engine of your backend. It handles security, calculates balances, and securely stores data in the SQL database.

3.2 Core Data Structures

This is a guaranteed viva question. You must know the difference between the four main data structures in Python: Lists, Tuples, Dictionaries, and Sets.

Mutability (The Most Important Concept)

Mutable means the data *can* be changed after it is created in memory. If you add an item to a list, it stays in the exact same memory location, it just gets bigger.

Immutable means the data *cannot* be changed. If you try to change a string in Python, the computer actually destroys the original string in memory and creates a brand new one. Strings, numbers, and Tuples are immutable.

1. Lists []

Lists are ordered, mutable, and allow duplicate values. Think of a shopping list. You can add items, remove items, and the order in which you wrote them down is preserved.

```
accounts = ["Savings", "Current", "Loan"]

accounts.append("Credit") # This works perfectly because
                           lists are Mutable.
```

2. Tuples ()

Tuples are ordered and allow duplicates, but they are **Immutable**. Once you create a tuple, you can never add or remove items from it.

- **Why use them?** Because they can't change, Python can process them much faster than lists. They also use less memory. You use tuples for data that should never be altered, like geographic coordinates (latitude, longitude) or RGB color codes.

3. Dictionaries { }

Dictionaries are unordered collections of Key-Value pairs. They are mutable. Think of an actual dictionary: you look up a word (the Key) to find the definition (the Value).

Dictionaries use **Hash Tables** under the hood. This means looking up a value takes almost zero time, even if the dictionary contains a million items. This is why we use dictionaries constantly in backend development.

```
user = {  
  
    "id": 101,  
  
    "name": "Salman",  
  
    "balance": 50000  
  
}  
  
print(user["name"]) # Instantly prints "Salman"
```

Viva Tip: An examiner might ask, "Can a list be a key in a dictionary?" The answer is NO. Dictionary keys must be **Immutable** (like strings or tuples) so that their internal hash value never changes.

4. Sets {}

Sets are unordered, mutable, but they **do not allow duplicates**. If you have a list of a thousand transaction IDs and you want to instantly remove all the duplicates, you simply convert the list into a Set.

3.3 Memory Management and Garbage Collection

In languages like C, developers have to manually allocate and free memory from the computer's RAM. If they forget, it causes a "memory leak" and the computer eventually crashes. Python handles this automatically.

Python uses **Reference Counting**. Imagine a Python object in memory is a balloon. Every variable that points to that object is a string holding the balloon. If three variables point to it, the reference count is 3. If you delete those variables, the strings are let go. When the reference count drops to 0, Python instantly pops the balloon (frees the memory).

Python also runs a background **Garbage Collector**. Sometimes, Object A points to Object B, and Object B points to Object A, creating an isolated circle. Even if your main program isn't using them, their reference count isn't zero. The Garbage Collector's job is to detect these isolated circles and destroy them.

3.4 Object-Oriented Programming (OOP)

OOP is a way of organizing code so that it maps to real-world concepts. Instead of writing random functions, you create "Classes" and "Objects".

- **Class:** The blueprint. Imagine architectural blueprints for a house. You can't live in a blueprint. It just defines what the house should look like (doors, windows, paint color).
- **Object (Instance):** The actual house built from the blueprint. You can build 100 different houses from the same blueprint. In code, `user1` and `user2` are two different objects created from the `User` class.

The `__init__` Method and `self`

In Python, the `__init__` method is the **constructor**. It is a special function that runs automatically the moment you create an object. It "initializes" the object's properties.

The `self` keyword is highly confusing to beginners. It simply refers to the specific object being created. If you have a blueprint instruction that says "paint the door red", you need to know *which* house's door to paint. `self.door = "red"` means "paint *my* door red".

```
class BankAccount:

    def __init__(self, owner, balance):

        self.owner = owner

        self.balance = balance

account1 = BankAccount("Salman", 1000) # __init__ runs automatically here!
```

3.5 Decorators (The '@' Symbol)

Since your project uses Flask, you use Decorators constantly (like `@app.route`). An examiner will definitely ask about this.

In Python, functions are "first-class citizens." This means a function is treated just like any other variable. You can pass a function into another function as an argument.

A Decorator is simply a function that takes another function, wraps it in some extra code, and returns the modified function. It allows you to "decorate" or modify a function without changing its internal code.

Example: In SmartBank, you might have a function called `view_dashboard()`. But you don't want anyone to see it unless they are logged in. You could write a massive if-statement inside the function. Or, you could write a decorator called `@login_required`. You place the decorator directly above the function. Before Python executes the dashboard logic, it runs the decorator's security check first.

3.6 Error Handling (Try / Except)

When bad things happen in code (like trying to divide by zero, or a database connection dying), Python throws an "Exception" and completely crashes the program. In a web server, a crash means the website goes offline for everyone.

To prevent this, we use Error Handling.

- **try:** Python attempts to run this block of code.
- **except:** If an error occurs inside the try block, Python immediately jumps here. Instead of crashing, you can log the error and return a safe message to the user.
- **finally:** This code runs *no matter what*. Even if an error happens, even if the function returns early. In backend development, this is absolutely crucial for databases. If your database query crashes, the finally block ensures that the database connection is safely closed, preventing server resource leaks.

Chapter 4: Databases and the Flask Framework

4.1 What is a Database?

If variables in JavaScript and Python act as memory boxes, why do we need a database? Because variables exist in RAM (Random Access Memory). When the server restarts or loses power, RAM is wiped clean. A database is a specialized software that saves data persistently to a hard drive so it survives reboots. For a banking app, losing data is not an option.

4.2 Relational (SQL) vs. Non-Relational (NoSQL)

There are two major families of databases in modern web development.

- **NoSQL (e.g., MongoDB):** Data is stored as flexible, JSON-like documents. You might have one document for a user that contains their name, age, and a nested list of their transactions. It's highly flexible and great for unstructured data or rapidly changing prototypes.
- **Relational SQL (e.g., SQLite, PostgreSQL):** Data is stored in strict, rigid tables with rows and columns. SmartBank uses a Relational Database. Why? Because banking data is highly structured and interconnected. A User has an Account. An Account has Transactions. SQL is built specifically to handle these relationships safely and efficiently.

4.3 The Anatomy of SQL Tables (Primary and Foreign Keys)

In your ``schema.sql`` file, you define tables. Think of a table like an Excel spreadsheet. Every column has a specific data type (e.g., INTEGER, TEXT).

The Primary Key

Every single row in a table must have a unique identifier. This is the **Primary Key**. In your ``users`` table, the ``id`` column is the Primary Key. Even if two users are named "Salman", their IDs will be ``1`` and ``2``. The database guarantees that no two rows will ever have the same Primary Key.

The Foreign Key

This is how relationships are formed. Suppose you have a ``transactions`` table. If you want to record that Salman deposited ₹500, you don't write "Salman" in the transactions table. You write ``1`` (Salman's Primary Key ID) in a column called ``user_id``. In this context, ``user_id`` is a **Foreign Key** because it points to the Primary Key of a foreign table. This links the two tables together.

4.4 SQL Commands (CRUD)

You interact with the database using SQL (Structured Query Language). Most operations fall under the acronym **CRUD**:

- **Create:** `INSERT INTO users (name, role) VALUES ('Salman', 'Admin')`
- **Read:** `SELECT * FROM users WHERE id = 1`

- **Update:** `UPDATE accounts SET balance = 5000 WHERE id = 1`
- **Delete:** `DELETE FROM transactions WHERE status = 'failed'`

4.5 ACID Properties (Crucial for Viva!)

For a banking application, the database must be ACID compliant. This guarantees that your money is safe even during server crashes.

- **Atomicity:** "All or Nothing." If you transfer ₹500 to a friend, the database must deduct ₹500 from you AND add ₹500 to your friend. If the server crashes in the exact millisecond between those two actions, Atomicity guarantees the database will automatically cancel the entire transaction. The money doesn't disappear.
- **Consistency:** The database rules are strictly enforced. If a column is defined as an INTEGER, trying to save the word "Hello" will be rejected instantly.
- **Isolation:** If two people try to transfer money at the exact same microsecond, Isolation ensures they are processed sequentially, so the math doesn't overlap and calculate the balance incorrectly.
- **Durability:** Once the database says "Transaction Successful", the data is permanently written to the hard drive. If someone kicks the server's power plug out a millisecond later, the data is still there when the server reboots.

4.6 The Flask Web Framework

Now that we have HTML/JS on the frontend and a Database on the backend, how do they talk? Enter **Flask**.

Flask is a web framework written in Python. A framework is simply a large collection of pre-written code that handles the boring, repetitive parts of web development (like processing HTTP headers and managing URLs), so you can focus on writing your actual app logic.

WSGI (Web Server Gateway Interface)

Python does not inherently know how to speak the language of the internet (HTTP). Web servers like Apache or Nginx speak HTTP, but they don't understand Python code. WSGI is the universal translator sitting in the middle. Flask uses Werkzeug, a WSGI utility library, to translate web requests into Python objects that your code can understand.

Micro-Framework vs. Batteries Included

You will likely be asked, "Why Flask and not Django?"

Django is a "batteries included" framework. It comes with its own massive ORM (database system), an admin panel, and an authentication system pre-installed. You just configure it.

Flask is a "micro-framework". It gives you the absolute bare minimum required to get a server running. Everything else—database connections, login logic—you have to write yourself. You chose Flask because it gave you absolute architectural freedom and allowed you to learn exactly how full-stack systems work from the ground up, rather than relying on automated magic.

Routing

Routing is how Flask knows which Python function to run when a user visits a specific URL.

```
@app.route('/dashboard', methods=['GET'])

def show_dashboard():

    return render_template('dashboard.html')
```

When the client's browser requests the URL `/dashboard`, Flask matches that URL to this specific decorator and executes the `show_dashboard()` function. The function then reads the HTML file from the hard drive and sends it back to the browser over the internet.

Chapter 5: SmartBank Code Walkthrough (Line-by-Line)

In your viva, the examiner won't just ask about theories. They will point to a file in your project and say, "Explain what this does." This chapter breaks down your actual code.

5.1 The Entry Point: `backend/main.py`

This is the file you run (`python main.py`) to start your entire server. It orchestrates everything.

```
from flask import Flask, request, jsonify

from flask_cors import CORS

from blueprints.auth_routes import auth_bp


app = Flask(__name__)

CORS(app)

app.secret_key = 'super_secret_key'


app.register_blueprint(auth_bp, url_prefix='/api/auth')


if __name__ == '__main__':

    app.run(debug=True, port=5000)
```

Line-by-Line Explanation:

- `from flask import Flask`: We import the main Flask class from the library we installed via pip.

- `from flask_cors import CORS`: **CORS (Cross-Origin Resource Sharing)** is a massive security feature. By default, browsers block a frontend running on `localhost:3000` from talking to a backend on `localhost:5000` to prevent hackers from stealing data. We import CORS to explicitly tell the browser, "It's okay, I own both of these, let them talk."
- `app = Flask(__name__)`: We create an instance (an Object) of the Flask class. This `app` variable is your web server.
- `CORS(app)`: We apply the CORS rules to our server.
- `app.secret_key = '...'`: Flask uses this cryptographic key to sign session cookies. If a user tries to forge a cookie to pretend they are the admin, Flask checks the signature against this key and rejects it.
- `app.register_blueprint(auth_bp, url_prefix='/api/auth')`: Instead of writing 10,000 lines of code in `main.py`, we split our app into modules (Blueprints). Here, we import the Authentication module and tell Flask: "Any time a user requests a URL starting with `/api/auth`, send them to this blueprint."
- `if __name__ == '__main__':`: This ensures the server only starts if we run `main.py` directly, not if we just import it into another script.
- `app.run(debug=True, port=5000)`: This starts the server on port 5000. `debug=True` means if our code crashes, it prints a helpful error message to the browser instead of a generic 500 Error. It also automatically restarts the server if we save changes to the code.

5.2 The Frontend Communication: user-auth.js

Now, let's look at what happens in the browser when the user clicks the "Login" button. We use Vanilla JavaScript.

```
async function handleLogin(event) {

    event.preventDefault();

    const username =
document.getElementById('loginUsername').value;

    const password =
document.getElementById('loginPassword').value;

    try {

        const response = await fetch('/api/auth/login', {

            method: 'POST',

            headers: { 'Content-Type': 'application/json' },

            body: JSON.stringify({ username: username,
password: password })

        });

        const data = await response.json();

        if (data.success) {

            window.location.href = 'userdash.html';
```

```
        } else {  
  
            alert(data.error);  
  
        }  
  
    } catch (error) {  
  
        console.error("Network failed", error);  
  
    }  
  
}
```

Line-by-Line Explanation:

- `async` function: We declare the function as asynchronous because it will be making a network request over the internet, and we don't want to freeze the browser while waiting.
- `event.preventDefault()`: By default, when you click a submit button inside an HTML form, the browser tries to refresh the entire page. We cancel this default behavior so we can handle the submission smoothly with JavaScript instead.
- `document.getElementById(...).value`: We use the DOM to grab the HTML input boxes by their unique IDs and extract the text the user typed inside them.
- `try { ... } catch (error) { ... }`: We wrap our network code in a try/except block. If the user loses their WiFi connection exactly when they click login, the `fetch` will fail. Instead of crashing the script, the catch block gracefully logs the network error.

- `await fetch('/api/auth/login')`: The **Fetch API** is built into modern browsers. It is how JS sends HTTP requests. We `await` it, meaning JS pauses execution on this specific line until the Python server replies, but allows the rest of the browser to keep running.
- `method: 'POST'`: We are sending data (username/password) to the server to create a session, so we use the HTTP POST method.
- `headers: { 'Content-Type': 'application/json' }`: We explicitly tell the Python server, "Hey, the data I'm sending you is formatted as JSON."
- `body: JSON.stringify(...)`: We take our JavaScript Object containing the username and password and convert it into a pure text JSON string so it can travel across the internet.
- `const data = await response.json()`: The Python server replies with a JSON string. We use `.json()` to parse that string back into a usable JavaScript Object.
- `if (data.success)`: If Python replied with `{"success": true}`, we use `window.location.href` to redirect the browser to the dashboard page.

5.3 The Database Logic: `core/db.py`

Finally, how does Python actually execute queries?

```
import sqlite3

from flask import g

def get_db():

    if 'db' not in g:

        g.db = sqlite3.connect('database/smartbank.db')

        g.db.row_factory = sqlite3.Row

    return g.db

def close_db(e=None):

    db = g.pop('db', None)

    if db is not None:

        db.close()
```

Line-by-Line Explanation:

- `import sqlite3`: We import the built-in library that allows Python to talk to SQLite databases.

- `from flask import g`: In Flask, `g` stands for **Global Context**. It is a special object that exists only for the duration of a single HTTP request. It's used to store temporary data that multiple functions might need while processing that one specific request.
- `if 'db' not in g::` We check if a database connection has already been opened for this user's request. Why? Because opening a database connection is slow and resource-heavy. We don't want to open 5 connections if we run 5 queries in a single request. This is called the **Singleton Pattern**.
- `g.db = sqlite3.connect(...)`: We open the connection to the database file on the hard drive and store the connection object in the `g` context.
- `g.db.row_factory = sqlite3.Row`: By default, SQLite returns rows as raw Tuples `(1, "Salman", 500)`. By setting this row factory, SQLite returns rows that act like Dictionaries `{"id": 1, "name": "Salman", "balance": 500}`. This makes the code vastly more readable because we can access columns by name instead of index numbers.
- `def close_db(e=None)::` This function is triggered by Flask the exact millisecond an HTTP request finishes and the response is sent to the user.
- `db = g.pop('db', None)`: We safely extract the database connection from the global context.
- `db.close()`: We explicitly close the connection. This is absolutely critical. If you don't close connections, the database eventually locks up and the entire server crashes.

Conclusion

You have now explored the absolute depths of full-stack engineering. You understand how networks communicate, how databases maintain integrity, and how the frontend and backend negotiate via APIs. Be proud of the system you've built. In your viva, speak slowly, use analogies, and demonstrate that you don't just know what the code does, but *why* it was written that way.